Building Rails SPAs in Frontend Ruby with Glimmer DSL for Web

wroclove.rb 2026

Andy Maleh • Senior Software Engineer • Eltropy Canada • MS in Software Engineering • RubyConf/RailsConf Speaker • Fukuoka Prefecture Future IT Initiative 2025 Award Winner (Glimmer DSL for Web) • Fukuoka Ruby 2022 Special Award Winner (Glimmer DSL for LibUI)

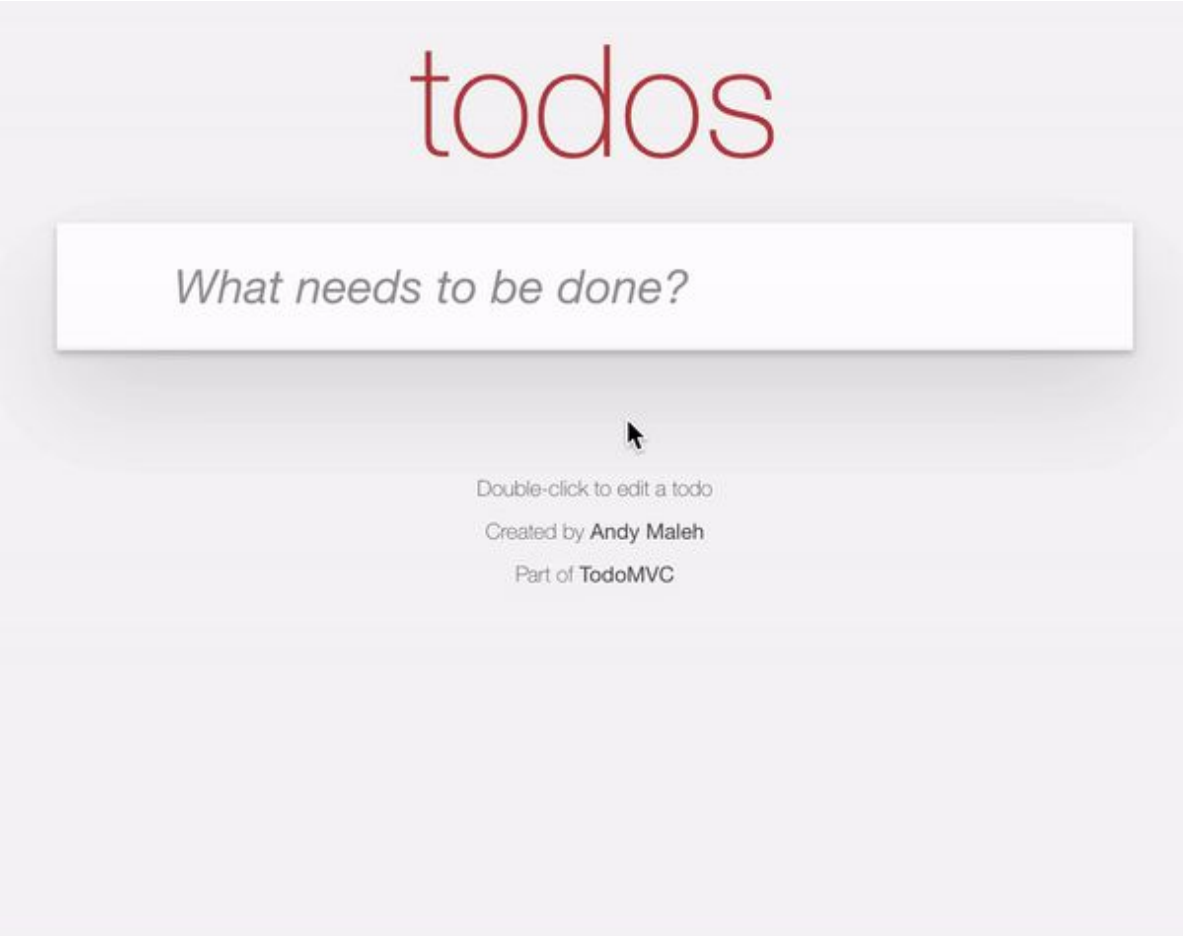
Glimmer DSL for Web

- Glimmer DSL for Web (Please "⭐" on GitHub): https://github.com/AndyObtiva/glimmer-dsl-web
- Ruby-in-the-Browser Web Frontend Framework for Rails to build SPAs (Single Page Applications) in Frontend Ruby
- Won Fukuoka Prefecture Future IT Initiative 2025 MoneyForward Award (international tech competition based in Japan) after being judged by Matz, the creator of Ruby, and other Fukuoka competition judges.

Glimmer DSL for Web

- Runs on Opal Ruby: https://opalrb.com/
  - Opal is a Ruby-to-JavaScript transpiler supporting Ruby 3.1+
  - Opal performance is similar to JavaScript's because it produces real JavaScript
  - Opal won a Fukuoka Ruby 2023 Award from Matz, creator of Ruby
- Glimmer DSL for Web is a drop-in replacement for JavaScript Frontend Development libraries like React, Angular, Ember, Vue, and Svelte. It provides a no-brainer upgrade in productivity and maintainability due to using Ruby instead of JS in the Frontend.

TodoMVC at https://sample-glimmer-dsl-web-rails7-app-black-sound-6793.fly.dev/



TodoMVC Example

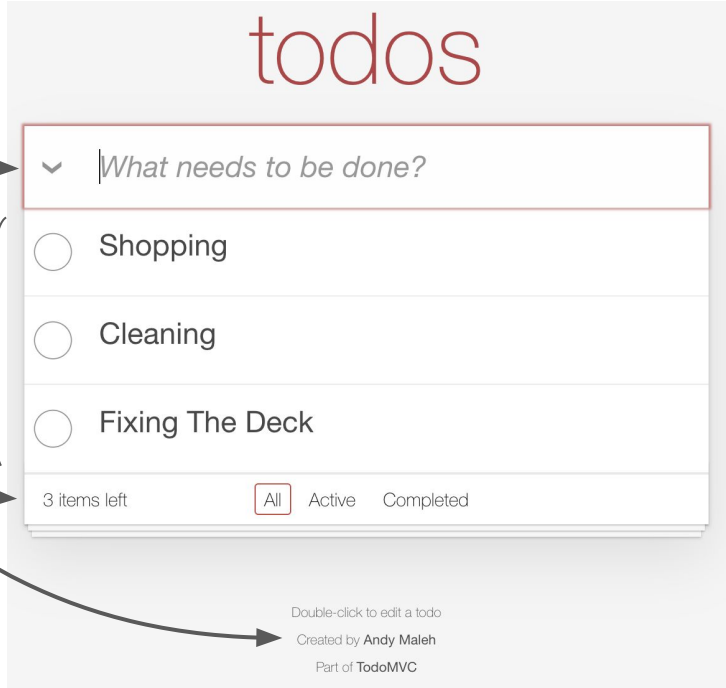
View

Model or Presenter

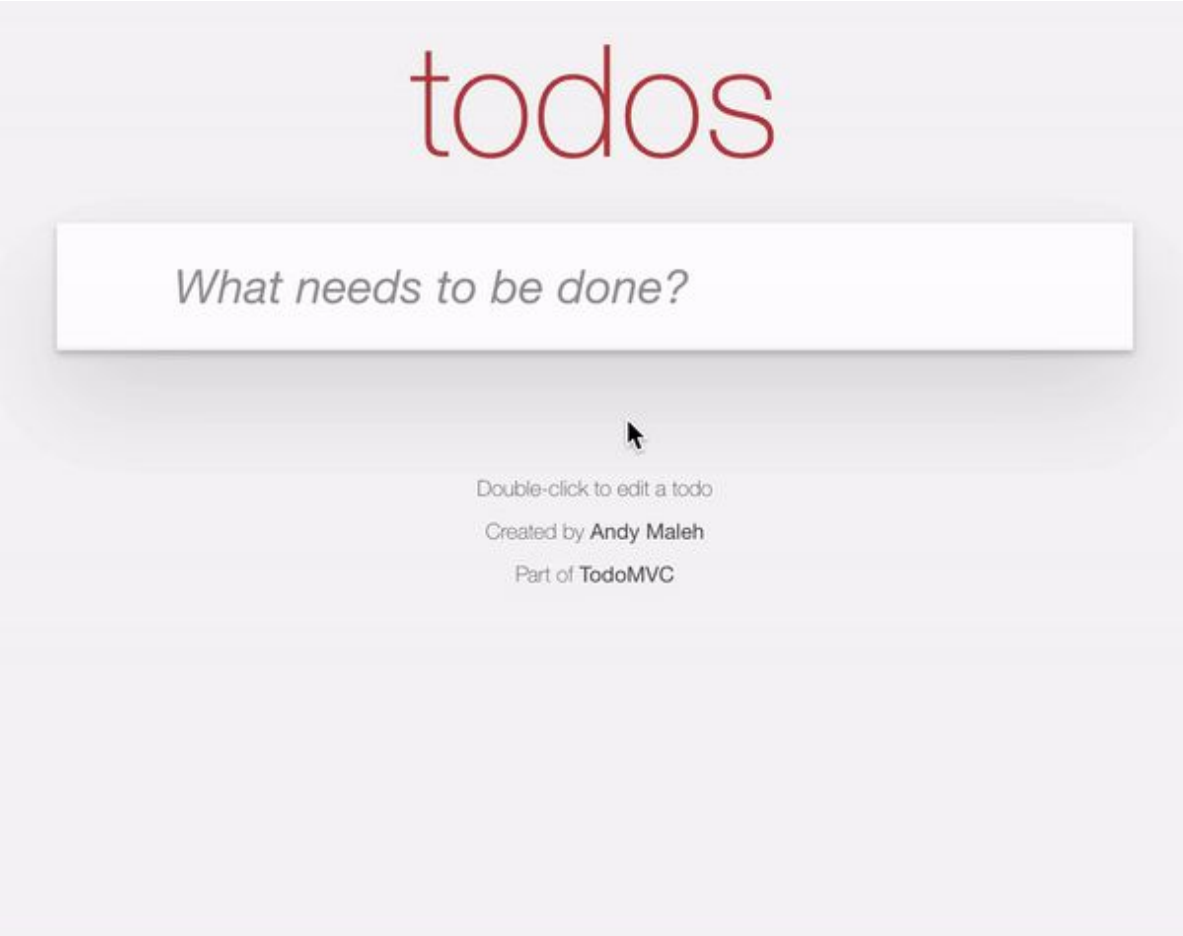
HTML

CSS

```
10  class TodoMvc
11    include Glimmer::Web::Component
12
13    before_render do
14      @presenter = TodoPresenter.new
15    end
16
17    after_render do
18      @presenter.setup_filter_routes
19    end
20
21    markup {
22      div {
23        section(class: 'todoapp') {
24          new_todo_form(presenter: @presenter)
25          todo_list(presenter: @presenter)
26          todo_filters(presenter: @presenter)
27        }
28        todo_mvc_footer
29
30        on_remove do
31          @presenter.unsetup_filter_routes
32        end
33      }
34    }
35
36    style {
37      r('body, button, html') {
38        margin 0
39        padding 0
40      }
41
42      r('button') {
43        _webkit_font_smoothing :antialiased
44        _webkit_appearance :none
```

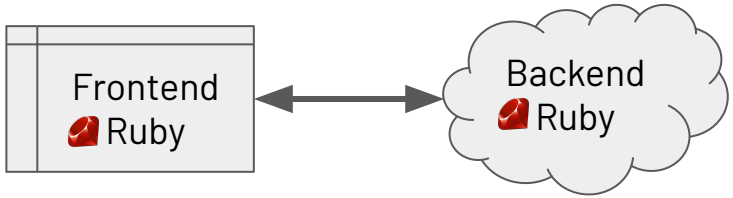


TodoMVC Code at github.com/AndyObtiva/glimmer-dsl-web/blob/master/lib/glimmer-dsl-web/samples/regular/todo_mvc.rb



Why Ruby in the Browser?

- Improve developer productivity with an isomorphic application architecture that enables using the same programming language on both the frontend and backend to minimize friction and maximize flow & creative problem solving.



Why Ruby in the Browser?

- Significantly better frontend code readability and maintainability via Ruby's unique syntax features, like **blocks** and **operator overloading**.

```
message = metadata << header << body << footer

message.handle_event 'sent' do |event|
  message.validate!
  message.deliver(event)
end

summary = messages.map(&:subject).join(' / ')
```

Why Ruby in the Browser?

- JavaScript is an awful and confusing programming language with very verbose syntax in spite of all the modern EcmaScript releases.

```
people.map((person) => person.address).
        filter((person) => person != null).
        filter((person) => person.isCanadian()).
        filter((person) => person.last_name == 'Smith').
        reduce((groups, person) => {
          groups[person.first_name] ||= []
          groups[person.first_name].push(person)
          return groups
        }, {})
```

Why Ruby in the Browser?

Ruby version of the same code we saw:

```
people.map(&:address).compact.select(&:canadian?).
       select {|person| person.last_name == 'Smith'}.group_by(&:first_name)
```

Why Ruby in the Browser?

- Ruby's Metaprogramming facilities are superior to those of JavaScript

```ruby
describe UsersController, type: :controller do
  before do
    allow(controller).to receive(:current_user).and_return(nil)
  end

  describe "GET #new" do
    subject { get :new }

    it "returns success" do
      expect(subject).to be_success
    end
  end
end
```

Why Ruby in the Browser?

- Ruby's ability to build declarative internal DSLs to write lightweight frontend GUI code that can mix logic unobtrusively, adopting a one language approach, instead of having to mix JavaScript & incompatible HTML/JSX/Templates.

```
div.food_ingredients {
  header.list_header {
    "Ingredients"
  }
  ul.food_items {
    ingredients.each_with_index do |name, index|
      classes = ['food_item']
      classes << 'highlighted' if index == 0
      li(id: "item_#{index}", class: classes) {
        name
      }
    end
  }
}
```

```
div.food_ingredients {
  header.list_header {
    "Ingredients"
  }
  ul.food_items {
    ingredients.each_with_index do |name, index|
      classes = ['food_item']
      classes << 'highlighted' if index == 0
      li(id: "item_#{index}", class: classes) {
        name
      }
    end
  }
}
```

```
<div class="food_ingredients">
  <header class="list_header">
    Ingredients
  </header>
  <ul class="food_items">
    <li id="item_0" class: "food_item highlighted">
      Carrot
    </li>
    <li id="item_1" class: "food_item">
      Cucumber
    </li>
    <li id="item_2" class: "food_item">
      Lettuce
    </li>
    <li id="item_3" class: "food_item">
      Onion
    </li>
    <li id="item_4" class: "food_item">
      Tomato
    </li>
  </ul>
</div>
```

Why Ruby in the Browser?

- Ruby has better core APIs than JavaScript, like the APIs for:
  - Array
  - Hash
  - Enumerable
  - String
  - Symbol
  - etc...

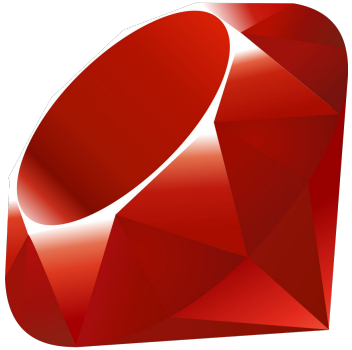
Why Ruby in the Browser?

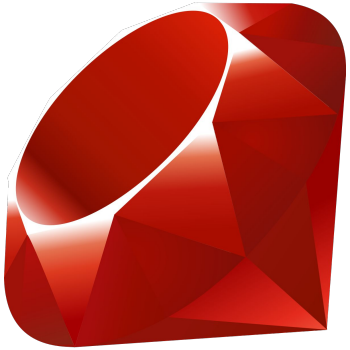
- Use JavaScript libraries from Ruby with enhanced Ruby code having higher readability, maintainability, and flexibility.

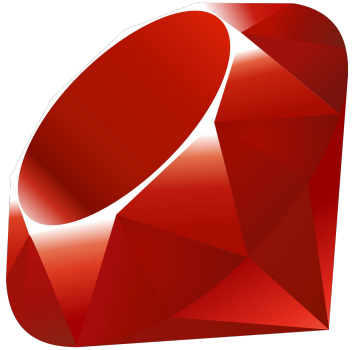
```
Document.ready? do
  non_inline_items = Element['.item'].reject {|e| e.css('display') == 'inline' }
  home_improvement_items = non_inline_items.select do |e|
    e.data('category') == 'Home Improvement'
  end
end
```

Why Ruby in the Browser?

- Reuse Ruby gems in the frontend.



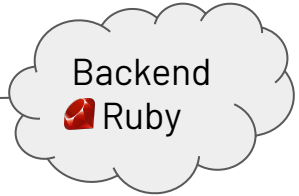




Why Ruby in the Browser?

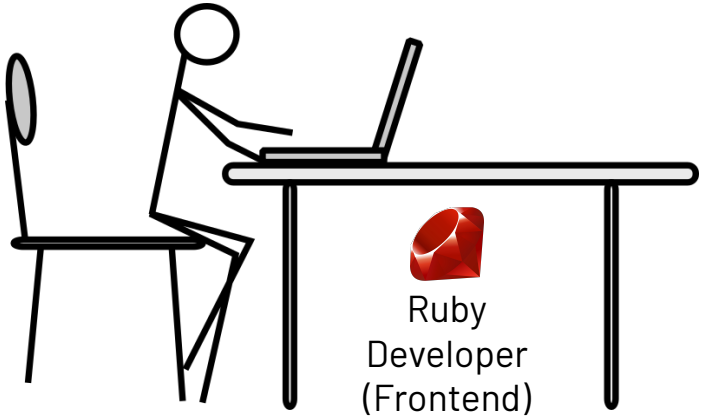
- Save development time by sharing/reusing Ruby code between the frontend and the backend.

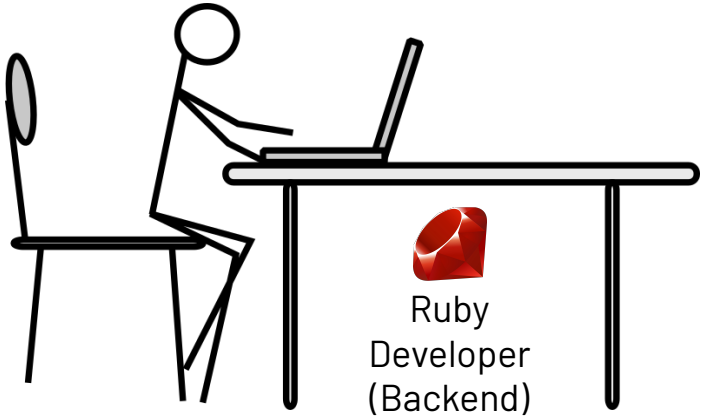
```ruby
class PriceCalculator
  def initialize(tax_index)
    @tax_index = tax_index
  end

  def calculate_with_tax(product, address)
    ...
  end

  def calculate_without_tax(product, address)
    ...
  end
end
```



Why Ruby in the Browser?

- Transfer developers' Ruby skills from the backend to the frontend.





Glimmer DSL for Web Design Principles

- The Ruby Way (including TIMTOWTDI: There Is More Than One Way To Do It)
- The Rails Way Convention over Configuration via smart defaults and automation of low-level details
- Requiring the least amount of code possible to build highly interactive web pages
- Declarative syntax that visually maps to the DOM (Document Object Model) hierarchy
- Ability to mix declarative and imperative code conveniently in one language
- Computers serve Software Engineers (not Software Engineers serve Computers)
- Think only about real world concepts directly relevant to web page interaction and application domain model
- Modular Software Design (e.g. support for Components)
- No premature optimization

Structure, Style, and Logic in One Language (Ruby)

Glimmer DSL for Web enables programming structure, style, and logic in one language, Ruby, improving productivity significantly:

- Logic is written with Opal (Ruby code)
- Structure is written with Glimmer HTML DSL (Ruby code)
- Style is written with Glimmer CSS DSL (Ruby code)

```
table(class: 'samples') {
  tbody {
    @presenter.samples.each do |sample|
      tr {
        td(sample.name)
        class_name <= [@presenter, :selected_sample,
                       on_read: -> (val) { val == sample ? 'selected' : '' }
                      ]
        onclick do |event|
          event.prevent_default
          @presenter.selected_sample = sample
        end
      }
    end
  }
  style {
    r('table.samples tr td') {
      border '1px solid transparent'
      padding @presenter.table_cell_padding
      cursor :default
    }
  }
}
```

Structure

Logic

Structure

Logic

Style スタイル

Glimmer DSL for Web Has Lighter Syntax than ERB (and HTML/JSX/Templates)

ERB ❌

```
<table>
  <tbody>
    <% @books.each do |book| %>
      <tr>
        <td><%= book.title %></td>
        <td><%= book.content %></td>
        <td><%= book_link(book) %></td>
        <% if book.editable? %>
          <td><%= edit_book_link(book) %></td>
        <% end %>
      </tr>
    <% end %>
  </tbody>
</table>
```

Glimmer ✅

```
table {
  tbody {
    @books.each do |book|
      tr {
        td(book.title)
        td(book.content)
        td { book_link(book) }
        if book.editable?
          td { edit_book_link(book) }
        end
      }
    end
  }
}
```

Glimmer Ruby HTML DSL

Glimmer DSL for Web supports building visual structure with a Ruby HTML DSL (Domain Specific Language), including SVG.

Ruby HTML DSL

```
html {
  head {
    title('Contact Manager')
  }
  body {
    form(action: '/search_contacts') {
      div {
        label('Name: ', for: 'name-field')
        input(type: 'text', id: 'name-field')
      }
      div {
        input(type: 'submit', value: 'Search')
      }
    }
  }
}
```

HTML

```
<html>
  <head>
    <title>Contact Manager</title>
  </head>
  <body>
    <form action="/search_contacts">
      <div>
        <label for="name-field" Name:   />
        <input type="text" id="name-field"  />
      </div>
      <div>
        <input type="submit" value="Search"  />
      </div>
    </form>
  </body>
</html>
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
html
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

head

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

body

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
form
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
input
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
label
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
select
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
option
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
table
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
thead
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
tbody
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
tr
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
th
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
td
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
h1
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

h2

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

h3

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
section
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
ol
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
ul
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
li
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
div
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

span

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

p

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
strong
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
em
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
address_form
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
contact_form
```

Glimmer Ruby HTML DSL Basics

- Glimmer Ruby HTML DSL enables building Web User Interfaces with a Ruby embedded (internal) Domain Specific Language
- 1) Element Keyword
  - Underscored case
  - Declarative control construction
  - Ruby method behind the scenes

```
contact_table
```

HTML Elements

- html
- head
- body
- form
- input
- label
- select
- option
- table

- thead
- tbody
- tr
- th
- td
- p
- ul
- li

Full list at: https://www.w3schools.com/tags/default.asp

Glimmer Ruby HTML DSL Basics

- 2) Element Arguments (Attributes + Text Content)
  - Optional
  - First argument can be String content if it fits on one line
  - Rest of arguments are Hash of HTML Attributes

```
h1('Contact Form')
```

Glimmer Ruby HTML DSL Basics

- 2) Element Arguments (Attributes + Text Content)
  - Optional
  - First argument can be String content if it fits on one line
  - Rest of arguments are Hash of HTML Attributes

```
h1('Contact Form', id: 'contact-form', class: 'header')
```

Glimmer Ruby HTML DSL Basics

- 2) Element Arguments (Attributes + Text Content)
  - Optional
  - First argument can be String content if it fits on one line
  - Rest of arguments are Hash of HTML Attributes

```
input(type: 'email', id: 'email-field', required: true)
```

Glimmer Ruby HTML DSL Basics

- 2) Element Arguments (Attributes + Text Content)
  - Optional
  - First argument can be String content if it fits on one line
  - Rest of arguments are Hash of HTML Attributes

```
label('Email: ', for: 'email-field')
```

Glimmer Ruby HTML DSL Basics

- 2) Element Arguments (Attributes + Text Content)
  - Optional
  - First argument can be String content if it fits on one line
  - Rest of arguments are Hash of HTML Attributes

```
input(type: 'submit', value: 'Add Contact')
```

Glimmer Ruby HTML DSL Basics

- 3) Element Content Block
  - Properties
  - Event Listeners
  - Nested Elements
  - String Content (return value)

```
table {
  ...
}
```

Glimmer Ruby HTML DSL Basics

- 3a) Element Content Block - Properties

```
div {
  inner_text article.content
}
```

Glimmer Ruby HTML DSL Basics

- 3a) Element Content Block - Properties

```
input(type: 'email', id: 'email-field') {
  value user.email
}
```

HTML DOM Element Properties

- id
- class_name
- inner_text
- inner_html
- style
- title
- dir
- lang
- scroll_left
- scroll_top
- tab_index

Full list at: https://www.w3schools.com/jsref/dom_obj_all.asp

Glimmer Ruby HTML DSL Basics

- 3b) Element Content Block - Event Listeners
  - Start with `on` prefix
  - Observer Pattern
  - Imperative code block

```
button('Greet') {
  onclick do
    $$.alert('Hi')
  end
}
```

Glimmer Ruby HTML DSL Basics

- 3b) Element Content Block - Event Listeners

```
input(type: 'submit', value: 'Add Contact') {
  onclick do |event|
    if [@name_input, @email_input].all?(&:check_validity)
      event.prevent_default
      presenter.add_contact
      @name_input.focus
    end
  end
}
```

HTML Event Listeners

- onblur
- onchange
- onclick
- oncopy
- oncut
- ondblclick
- ondrag
- ondrop
- onfocus
- oninput
- onkeydown
- onkeyup
- onmousedown
- onmouseup
- onmouseenter
- onmouseleave
- onscroll
- ontoggle

Full list at: https://www.w3schools.com/jsref/dom_obj_event.asp

Glimmer Ruby HTML DSL Basics

- 3c) Element Content Block - Nested Elements
  - Declarative Hierarchy
  - Composite Pattern

```
table {
  tr {
    td('John Doe')
    td('johndoe@example.com')
  }
}
```

Glimmer Ruby HTML DSL Basics

- 3c) Element Content Block - Nested Elements

```
form {
  div {
    label('Name: ', for: 'name-field')
    input(type: 'text', id: 'name-field')
  }
  div {
    label('Email: ', for: 'email-field')
    input(type: 'email', id: 'email-field')
  }
}
```

Glimmer Ruby HTML DSL Basics

- 3d) Element Content Block - String Content
  - Block return value
  - Must be last

```
h1(class: 'title') {
  'Flying Cars Become Reality!'
}
```

Glimmer Ruby HTML DSL Basics

- 3d) Element Content Block - String Content
  - Block return value
  - Must be last

```
h1('Flying Cars Become Reality!', class: 'title')
```

Glimmer Ruby HTML DSL Basics

- 3d) Element Content Block - String Content
  - Block return value
  - Must be last

```
p {
  "Travel-X came up with a new feature for their flying car
  the Ptero, which relies on AI to auto-balance in mid-air,
  significantly facilitating piloting by average consumer."
}
```

Glimmer Ruby HTML DSL Basics

- 3d) Element Content Block - String Content
  - Block return value
  - Must be last

```
p {"
  #{strong('Travel-X')} came up with a new feature for flying
  car, #{em('Ptero')}, which relies on AI to auto-balance in
  Mid-air, facilitating piloting by average consumer.
"}
```

Glimmer Ruby HTML DSL Basics

- 4) Element Methods
  - Invoked through underscored Ruby Methods
  - Match HTML DOM Element API
  - Proxy calls to wrapped HTML DOM objects
  - Include access to element properties after construction

```
@email_input = input(type: 'email', required: true)
email_is_valid = @email_input.check_validity
email_value = @email_input.value
@email_input.focus
```

Glimmer Ruby HTML DSL Basics

- 4) Element Methods
  - `content {}` is a special Glimmer method for re-opening element content block and adding more content like sub-elements

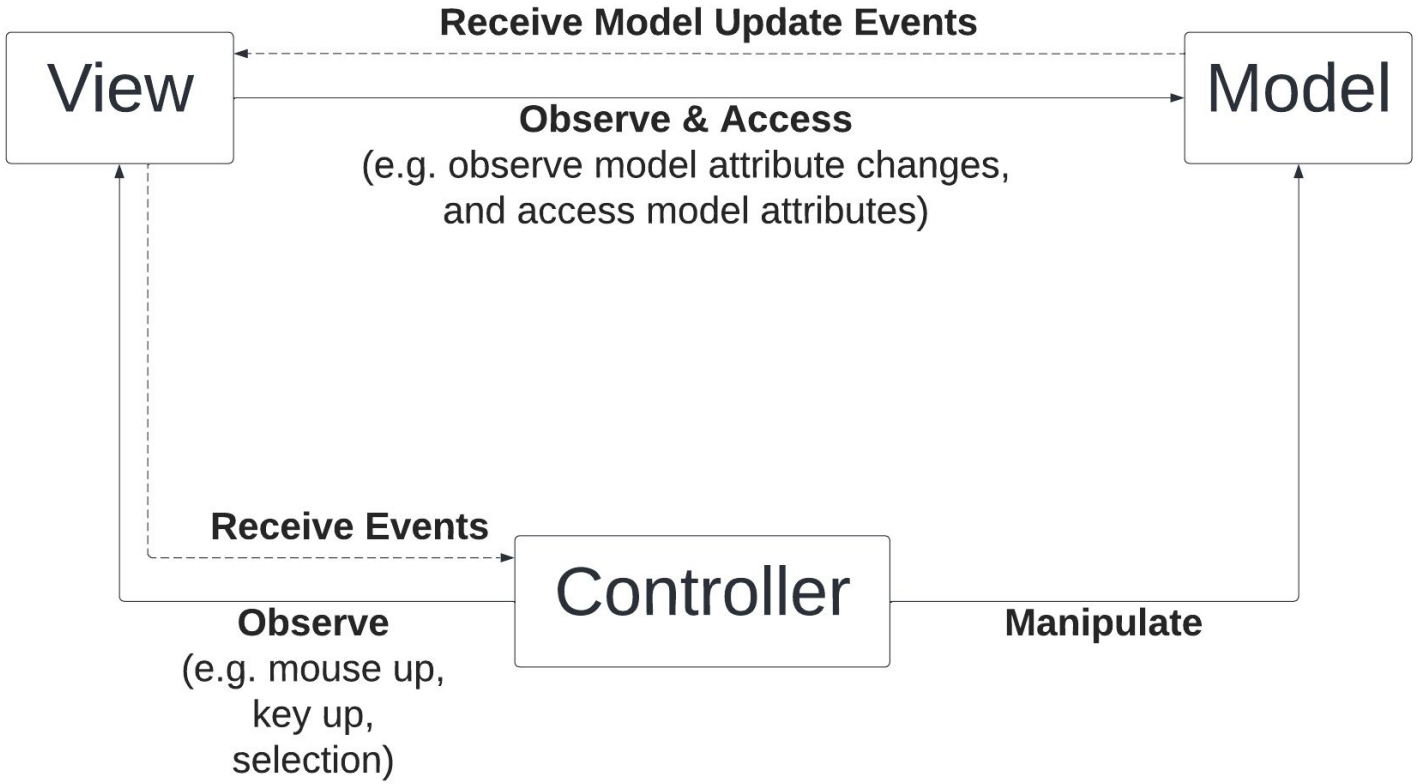
```
@table.content {
  tr {
    td('John Doe')
    td('johndoe@example.com)
  }
}
```

HTML DOM Element Methods

- content
- empty
- check_validity
- blur
- focus
- offset_height
- offset_width
- offset_left
- offset_top
- scroll_height
- scroll_width
- scroll_left
- scroll_top

Full list at: https://www.w3schools.com/jsref/dom_obj_all.asp

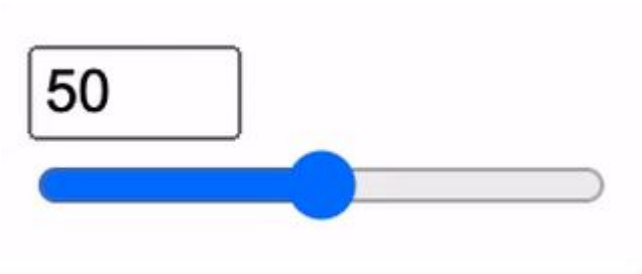
MVC Software Architecture



Observer Pattern

- Observer Design Pattern is foundational in MVC Architectural Pattern
- In MVC (Model-View-Controller):
  - Controller observes View for user actions (e.g. mouse/keyboard) and trigger changes in Model
  - View observes Model for domain data changes and updates itself
- In Glimmer GUI DSL (Graphical User Interface Domain Specific Language):
  - Controller can rely on Listeners to observe View actions and trigger changes in Model
  - View can rely on **`observe(model, attribute)`** method to observe a Model attribute for changes and update its controls accordingly. It relies on Ruby Meta-Programming to auto-enhance a Model into an **`ObservableModel`**

Hello, Observer! - Example



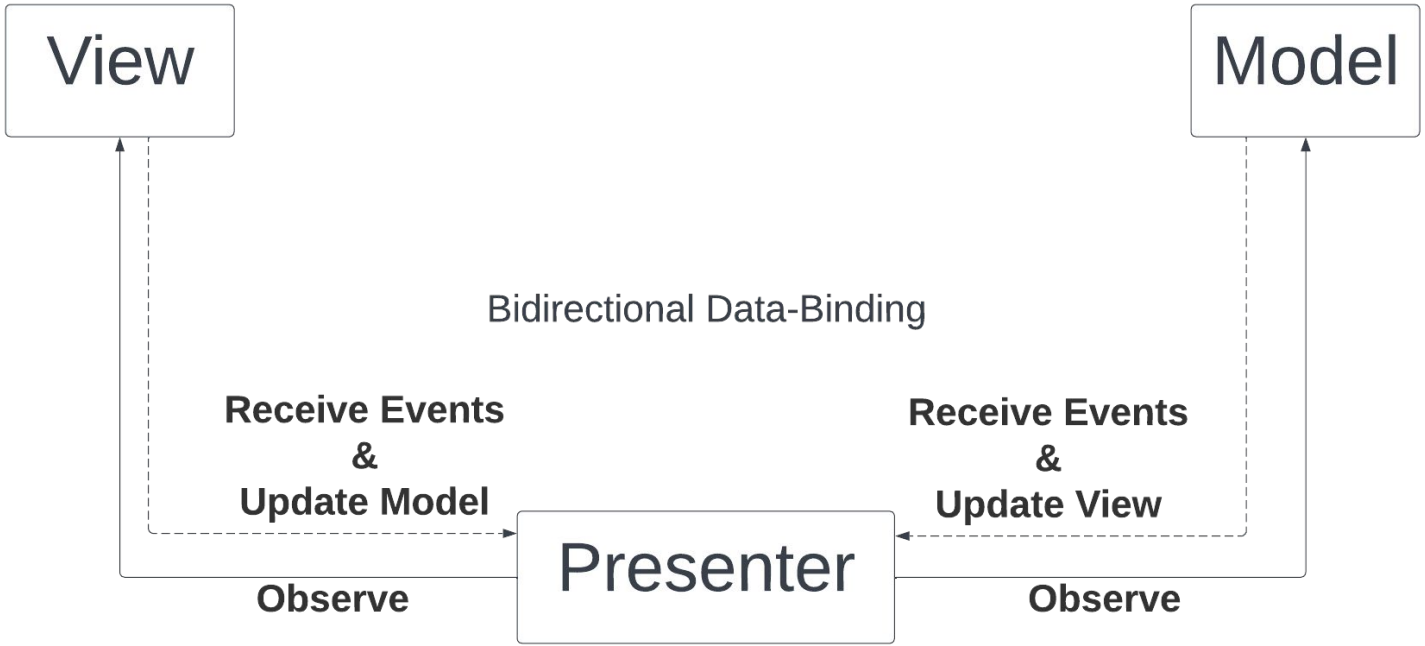
Hello, Observer! - Model

```
1  class NumberHolder
2    attr_accessor :number
3
4    def initialize
5      self.number = 50
6    end
7  end
```

Hello, Observer! - View Component

```ruby
class HelloObserver
  include Glimmer::Web::Component

  before_render do
    @number_holder = NumberHolder.new
  end

  after_render do
    @number_input.value = @number_holder.number
    @range_input.value = @number_holder.number
    observe(@number_holder, :number) do
      number_string = @number_holder.number.to_s
      @number_input.value = number_string unless @number_input.value == number_string
      @range_input.value = number_string unless @range_input.value == number_string
    end
  end

  markup {
    div {
      div {
        @number_input = input(type: 'number', min: 0, max: 100) {
          oninput do
            @number_holder.number = @number_input.value.to_i
          end
        }
      }
      div {
        @range_input = input(type: 'range', min: 0, max: 100) {
          oninput do
            @number_holder.number = @range_input.value.to_i
          end
        }
      }
    }
  }
end
```

MVP & Data-Binding Software Architecture



Property Data-Binding

- Property Data-Binding declaratively & automatically synchronizes View properties w/ Model attributes bidirectionally or unidirectionally
- Bidirectional (two-way) with **<=>** for View read/write properties

```
input(type: 'text') {
  value <=> [contact, :name]
}

input(type: 'email') {
  value <=> [contact, :email]
}
```

Property Data-Binding

- Unidirectional (one-way) with **<=** for View read-only properties, inline style data-binding, and CSS class-inclusion data-binding

```
div {
  inner_text <= [address, :text]
}
```

Property Data-Binding

- Unidirectional (one-way) with **<=** for View read-only properties, inline style data-binding, and CSS class-inclusion data-binding

```
button {
  inner_text <= [@counter, :count,
                  on_read: ->(value) { "Count: #{value}" }
                ]
}
```

Property Data-Binding (Inline Style)

- Unidirectional (one-way) with **<=** for View read-only properties, inline style data-binding, and CSS class-inclusion data-binding

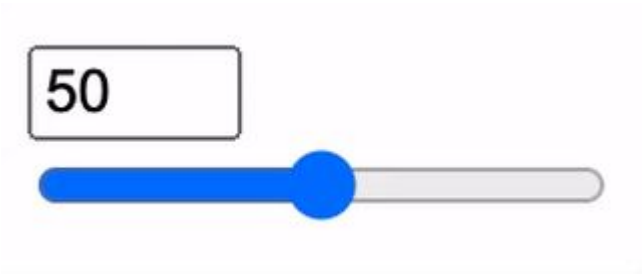
```
style(:width) <= [button_model, :width]
style(:height) <= [button_model, :height]
style(:font_size) <= [button_model, :font_size]
style(:background_color) <= [button_model, :background_color]
style(:border_color) <= [button_model, :border_color,
                         computed_by: :background_color]
```

Property Data-Binding (CSS Class-Inclusion)

- Unidirectional (one-way) with **<=** for View read-only properties, inline style data-binding, and CSS class-inclusion data-binding

```
class_name(:pushed) <= [button_model, :pushed]
class_name(:pulled) <= [button_model, :pushed, on_read: :!]
```

Hello, Observer! - Example w/ Data-Binding



Hello, Observer! - Model

```
1  class NumberHolder
2    attr_accessor :number
3
4    def initialize
5      self.number = 50
6    end
7  end
```

Hello, Observer! - View Component w/ Data-Binding

```
1  class HelloObserver
2    include Glimmer::Web::Component
3
4    before_render do
5      @number_holder = NumberHolder.new
6    end
7
8    markup {
9      div {
10       div {
11         input(type: 'number', min: 0, max: 100) {
12           value <=> [@number_holder, :number]
13         }
14       }
15       div {
16         input(type: 'range', min: 0, max: 100) {
17           value <=> [@number_holder, :number]
18         }
19       }
20     }
21   }
22 end
```

Content Data-Binding

- Content Data-Binding declaratively & automatically generates (re-renders) View content in response to Model attribute changes
- Under an element, nest dynamically generated content inside an explicit **`content`** block that specifies the model and attribute to observe:

```
content(model, attribute) {
    … # controls to generate dynamically
}
```

Hello, Content Data-Binding!

Number of addresses: 1

Name

Street

City

State

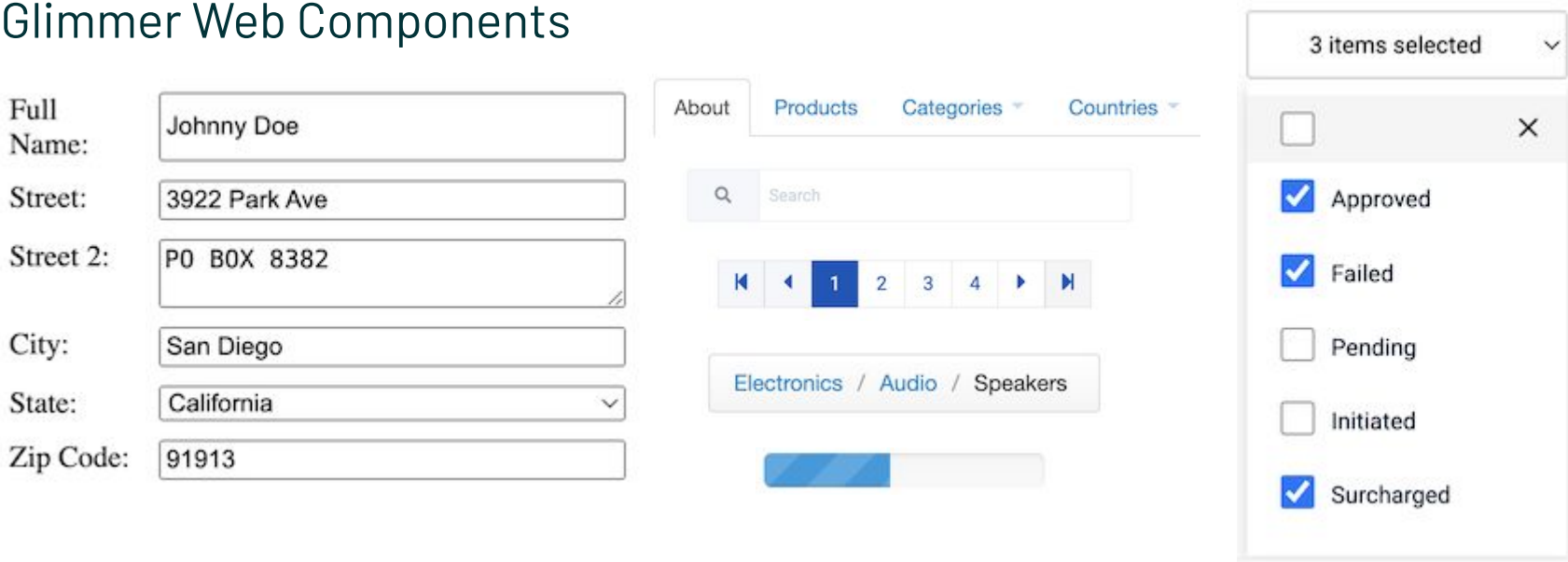
Zip

Hello, Content Data-Binding! View Code Excerpt

```
1  div {
2    div {
3      label('Number of addresses: ', for: 'address-count-field')
4      input(id: 'address-count-field', type: 'number', min: 1, max: 3) {
5        value <=> [@user, :address_count]
6      }
7    }
8
9    div {
10     content(@user, :addresses) do
11       @user.addresses.each do |address|
12         div {
13           div(style: 'display: grid; grid-auto-columns: 80px 280px;') { |address_div|
14             [:name, :street, :city, :state, :zip].each do |attribute|
15               label(attribute.to_s.capitalize, for: "#{attribute}-field")
16               input(id: "#{attribute}-field", type: 'text') {
17                 value <=> [address, attribute]
18               }
19             end
20
21             div(style: 'grid-column: 1 / span 2;') {
22               inner_text <= [address, :text]
23             }
24
25             ... # styles
26           }
27         }
28       end
29     end
30   }
31 }
```

Glimmer Web Components

Glimmer DSL for Web supports building custom visual components with Glimmer Web Components



```ruby
class AddressForm
  include Glimmer::Web::Component

  option :address

  markup {
    div {
      div(style: 'display: grid; grid-auto-columns: 80px 260px;') {
        label('Full Name: ', for: 'full-name-field')
        input(id: 'full-name-field') {
          value <=> [address, :full_name]
        }

        label('Street: ', for: 'street-field')
        input(id: 'street-field') {
          value <=> [address, :street]
        }

        label('Street 2: ', for: 'street2-field')
        textarea(id: 'street2-field') {
          value <=> [address, :street2]
        }

        label('City: ', for: 'city-field')
        input(id: 'city-field') {
          value <=> [address, :city]
        }

        label('State: ', for: 'state-field')
        select(id: 'state-field') {
          Address::STATES.each do |state_code, state|
            option(value: state_code) { state }
          end

          value <=> [address, :state_code]
        }
        ...
```

```ruby
class AddressForm
  include Glimmer::Web::Component

  option :address

  markup {
    div {
      div(style: 'display: grid; grid-auto-columns: 80px 260px;') {
        label('Full Name: ', for: 'full-name-field')
        input(id: 'full-name-field') {
          value <=> [address, :full_name]
        }

        label('Street: ', for: 'street-field')
        input(id: 'street-field') {
          value <=> [address, :street]
        }

        label('Street 2: ', for: 'street2-field')
        textarea(id: 'street2-field') {
          value <=> [address, :street2]
        }

        label('City: ', for: 'city-field')
        input(id: 'city-field') {
          value <=> [address, :city]
        }

        label('State: ', for: 'state-field')
        select(id: 'state-field') {
          Address::STATES.each do |state_code, state|
            option(value: state_code) { state }
          end

          value <=> [address, :state_code]
        }
        ...
```

```ruby
div {
  h1('Shipping Address')

  address_form(address: @shipping_address)

  h1('Billing Address')

  address_form(address: @billing_address)
}
```

Shipping Address

Full Name: Johnny Doe

Street: 3922 Park Ave

Street 2: PO BOX 8382

City: San Diego

State: California

Zip Code: 91913

Billing Address

Full Name: John C Doe

Street: 123 Main St

Street 2: Apartment 3C

City: San Diego

State: California

Zip Code: 91911

```
div {
  h1('Shipping Address')

  address_form(address: @shipping_address)

  h1('Billing Address')

  address_form(address: @billing_address)
}
```

Glimmer Web Components

- **`include Glimmer::Web::Component`**
  - Generates new Glimmer Ruby HTML DSL keyword matching underscored name of class.
  - e.g. AddressForm class generates **`address_form`** keyword
- Define **`options`** (attributes) that can be passed as **`Hash`** arguments
- Define **`markup`** block content with Glimmer Ruby HTML DSL
- **`before_render`** block hook to setup variables like models & presenters
- **`after_render`** block hook to register event listeners

Glimmer Web Components

- ComponentClass.**`render`** method renders & mounts component in HTML document body by default
- Passing `**`parent: css_selector`**` option mounts element somewhere other than in body
  - e.g. **`SampleSelector.render(parent: "#container")`**
- **`markup_root`** method can be used inside ComponentClass to get a reference of the top-level element at the root of the **`markup`** block

Glimmer Web Component Slots

- Slots enable customizing components following the Open/Closed Principle (Open for extension / Closed for modification)
- Make any HTML element a slot by passing (slot: :slot_name) option
- Now, you can nest the slot name as a virtual element within a component, and it will fill the component in the HTML element that is made a slot.

Glimmer Web Component Slots

- Example:

```
address_form(address: @shipping_address) {
  address_header {
    p('This is the address that is used for shipping your purchase.')
  }
  address_footer {
    p('Thank you for
  }
```

Shipping Address

This is the address that is used for shipping your purchase.

Full Name:	Johnny Doe
Street:	3922 Park Ave
Street 2:	PO BOX 8382

Glimmer Web Component Default Slot

- For components with slots, if a specific slot is used often by consumers, it can be designated as the default slot, so consumers can write less code by nesting content directly under component without specifying default slot explicitly
- Declare default slot with class method invocation: `default_slot :section_content`
- Now, consumers do not need to open a default slot block (e.g. `section_content {}`) anymore. They can nest content directly under the component.

Glimmer Web Component Default Slot

- Example:

```
address_form(address: @shipping_address) {
  p('This is the address that is used for shipping your purchase.')
  ...
}
```

Shipping Address

This is the address that is used for shipping your purchase.

Full Name:	Johnny Doe
Street:	3922 Park Ave
Street 2:	PO BOX 8382

Glimmer Web Component Custom Listeners

- Components can support custom listeners by declaring events (e.g. events :expanded, :collapsed)
- Consumers can hook into component custom listeners by calling a listener with the prefix 'on_' and suffix as the event name
  - Example:
    - on_expanded { do_something_upon_expansion}
    - on_collapsed { do_something_upon_collapse }

Glimmer Web Component Custom Listeners

- Example: an accordion component can listen to the expanded and collapsed events of an accordion_section sub-component to ensure that only one section is expanded at any time

Glimmer Web Component Attribute Data-Binding

- Component attributes (options) can be data-bound to Models/Presenters just like how HTML element attributes can be data-bound to Models/Presenters
- That updates the View automatically whenever Models/Presenters change
- Prevents the need to re-render a View if not needed, simplifying code significantly

Glimmer Web Component Attribute Data-Binding

- Example:

```
address_form {
  address <=> [presenter, :shipping_address]
  ...
}
```

Shipping Address

This is the address that is used for shipping your purchase.

Full Name:	Johnny Doe
Street:	3922 Park Ave
Street 2:	PO BOX 8382

glimmer_component Rails Helper

- **`glimmer_component`** Rails helper method enables rendering a Glimmer Web Component within a backend ERB View file
- To use, **`include GlimmerHelper`** in **`ApplicationHelper`** or the Rails helper of the controller that is rendering the ERB View
- The component is rendered inside the element in which **`glimmer_component`** Rails helper was used, enabling handling part of the page with a Glimmer Web Component

```
<div id="address-container">
  <h1>Shipping Address </h1>
  <legend>Please enter shipping address</legend>
  <%= glimmer_component('address_form',
        full_name: params[:full_name],
        street: params[:street],
        street2: params[:street2],
        city: params[:city],
        state: params[:state],
        zip_code: params[:zip_code]
      )
   %>
   <div>
     <a href="/" data-turbo="false">&lt;&lt; Back Home</a>
   </div>
</div>
```

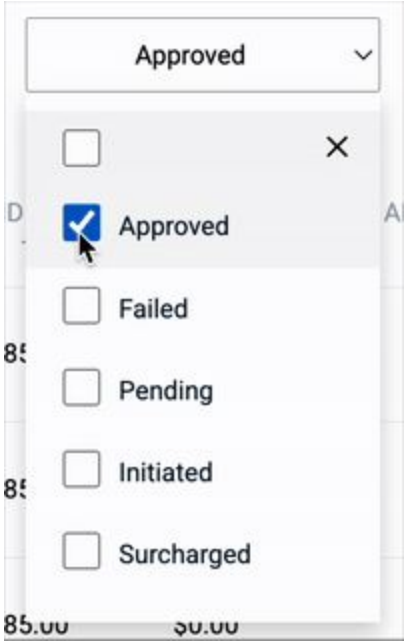
glimmer-web-components Ruby Gem

- Glimmer Web Components (Please "⭐" on GitHub):

  https://github.com/AndyObtiva/glimmer-web-components

- **glimmer-web-components** is a Ruby Gem that aims to provide reusable Glimmer Web Components that are extracted from my job's Rails web app.
- Example: `multi_checkbox_dropdown`

```
multi_checkbox_dropdown(values: SomePresenter::STATUS_FILTER_TYPES) {
  selected_values <=> [@some_presenter, :status_filters]
}
```



Glimmer Paragraph Formatting Elements

- To facilitate building formatted textual paragraphs in Ruby, Glimmer DSL for Web behaves differently under the **`p`** element (paragraph) when using formatting elements (below), rendering strings immediately to enable String concatenation:
- **`br, strong, em, br, i, sub, sup, del, ins, small, mark`**

Hello, Paragraph!

Flying Cars Become 100% Safe with AI Powered Balance!

In the early 2030's, *flying cars* became affordable after their prices dropped below ~~$100,000~~$80,000 as a result of the innovations of **Travel-X**. Still, that did not make *flying cars* any popular due to the extreme difficulty in piloting such flying vehicles for the average person, making it very tough to pass the tests for getting a piloting license given the learning curve.

That said, **Travel-X** has recently come up with a new feature for their flagship *flying car*, the Ptero[1]™, which relies on AI[2] to automatically balance the flying cars in mid-air, thus significantly facilitating their piloting by the average consumer.

That Ptero™ will be so stable and well balanced while flying that the consumer will be able to drive as if it is a plain old car, with the only difference being vertical elevation, the control of which will be handled automatically by AI. The Ptero™ will debut for $79,000.

Legend:

1- Ptero: Pterosaur is flying dinosaur species
2- AI: Artificial Intelligence

```
class HelloParagraph
  include Glimmer::Web::Component

  markup {
    div {
      h1(class: 'title') {
        'Flying Cars Become 100% Safe with AI Powered Balance!'
      }

      p(class: 'intro') {"
        In the early 2030's, #{em('flying cars')} became affordable after their prices dropped
        below #{small(del('$100,000'))}#{ins('$80,000')} as a result of the innovations of #{strong('Travel-X')}. Still, that did not
        make #{em('flying cars')} any popular due to the extreme difficulty in piloting such flying vehicles for the average
        person, making it very tough to pass the tests for getting a piloting license given the learning curve.
      "}

      p {"
        That said, #{b('Travel-X')} has recently come up with a new feature for their flagship #{i('flying car')},
        the Ptero#{sub(1)}#{sup('TM')}, which relies on AI#{sub(2)} to automatically balance the flying cars in mid-air,
        thus significantly facilitating their piloting by the average consumer.
      "}

      p(class: 'conclusion') {"
        That Ptero#{sup('TM')} will be so stable and well balanced while flying that the consumer will be able to drive
        as if it is a plain old car, with the only difference being vertical elevation, the control of which will be handled
        automatically by AI. The Ptero#{sup('TM')} will debut for #{span(style: 'text-decoration: underline dashed;'){'$79,000'}}.
      "}

      h2(class: 'legend-title') {
        mark('Legend:')
      }

      p(class: 'legend') {"
        #{strong("1- Ptero:")} Pterosaur is flying dinosaur species#{br}
        #{strong("2- AI:")} Artificial Intelligence#{br}
      "}

    }
  }
end
```

CSS in Glimmer DSL for Web

- Glimmer DSL for Web can work with 3 CSS options:
  - Built-in Glimmer Ruby CSS DSL (fast enough as CSS styles are automatically added to HTML head as per web recommendations)
  - External CSS/SCSS styles (Glimmer produces standard HTML), including integration with libraries like Tailwind/Bootstrap
  - Hybrid Glimmer CSS DSL + External CSS/SCSS Approach
    - General styles are handled in external CSS/SCSS files
    - Specific styles are handled in Glimmer CSS DSL

Glimmer Ruby CSS DSL

Glimmer DSL for Web supports styling elements with a Ruby CSS DSL (Domain Specific Language).

Ruby CSS DSL

```
body {
  font_size 1.1.em
  background :white
  width 90.%
  height 100.%
}
media('screen and (max-width: 600px)') {
  rule('body#app h1.title') {
    font_size 16
    font_family '"Times New Roman", serif'
  }
}
```

CSS

```
body {
  font-size: 1.1em;
  background: white;
  width: 90%;
  height: 100%;
}

@media screen and (max-width: 600px) {
  body#app h1.title{
    font-size: 16px;
    font-family: "Times New Roman", serif
  }
}
```

Glimmer Ruby CSS DSL

- Glimmer CSS DSL enables writing CSS in Ruby with logic (e.g. if/each)
- **`style`** element can contain Glimmer DSL for CSS syntax
- **`rule(selector) { expressions }`** :
  - **`selector`** includes a CSS selector like **`#id`** or **`.class`**
  - **`expressions`** is one or more CSS expressions explained below
- CSS **`expression`** consists of a property & value separated by space:
  - CSS property: any CSS property, but dashes become underscores
  - String value: any CSS value, but in String form
- **`r`** / **`ru`** / **`rul`** can be used as short aliases for **`rule`**

Glimmer CSS DSL Example

```
style {
  r('.main') {
    border_top '1px solid #e6e6e6'
    position :relative
    z_index '2'
  }

  r('.toggle-all') {
    border :none
    bottom 100.%
    height 1
    opacity '0'
    position :absolute
    right 100.%
    width 1
  }
  ...
}
```

Web Requests

- Glimmer DSL for Web can handle Web Requests in multiple ways:
  - `Rails::ResourceService` supports Rails RESTful CRUD operations like index, show, create, update, destroy. Only needs a resource Model argument at minimum. It can figure out the rest automatically from the Model name (like what URL to hit)
  - opal-jquery gem is included in Glimmer DSL for Web and provides basic jQuery Ajax support via the `HTTP` class
  - Reuse a JS web request library like Axios
  - Use built-in JS `fetch` from Opal

Web Requests via Rails::ResourceService

- Example:

```
class ContactPresenter
  attr_accessor :contacts, :edit_index

  ...

  def update_contact
    Rails::ResourceService.update(resource: form_contact) do |response, updated_contact, errors|
      if response.ok?
        contacts[edit_index].load_with(updated_contact)
        self.edit_index = nil
        form_contact.reset
        form_contact.errors = nil
      else
        form_contact.errors = errors
      end
    end
  end
```

Contact Manager

First Name	Bob	Street	38 Derek Ave
Last Name	Gibbons	City	Janebrook
Email	bob@gibbons.ca	State	Washington
Phone	837-822-8372	Zip	98001

Update Contact

First Name	Last Name	Email	Phone	Street	City	State	Zip	
Josh	Harvard	josh@harvy.com	206-738-2324	8322 Ender Rd	Seattle	Washington	98001	
Sean	Gibbons	sean@gibs.net	382-836-2938	223 Main St	Los Angeles	California	90928	
Maria	Brick	mar@bri.io	322-322-8383	128 Corner Ave	Denver	CO	80003	
Bob	Gibbons	bob@gibbons.ca	837-822-8372	38 Derek Ave	Janebrook	Washington	98001	✕
Sofia	Alvarez	sofia@alvarez.net	518-832-8362	3298 Stevenson	Austin	Texas	73308	
artur	komarov	artur.komarov@gmail.com			Moscow			
Test	Joke	Test@hehe.com	555-835-4526	101 RiverDry	RockBottom	MS	37234	

Web Requests via Ajax with opal-jquery HTTP

- Glimmer DSL for Web includes opal-jquery gem, which supports making Ajax calls using jQuery via the HTTP class with the same high-level APIs:
  - `HTTP.get(url, options) { |response| … }`
  - `HTTP.post(url, options) { |response| … }`
  - `HTTP.put(url, options) { |response| … }`
  - `HTTP.delete(url, options) { |response| … }`
  - `HTTP.patch(url, options) { |response| … }`
  - `HTTP.head(url, options) { |response| … }`
- `response.body` must be wrapped by `native()` to call attributes

Ajax with opal-jquery HTTP Example

```
HTTP.get("/samples/#{selected_sample}.json") do |response|
  code = Native(response.body).code
  self.selected_sample_code = code
end
```

Integration with JavaScript Libraries

Glimmer DSL for Web supports integration with any JavaScript libraries using the Opal Ruby require statement or HTML script tag with src attribute.

```
require 'vendor/jquery'
```

Opal Ruby require statement

```
script(src: 'https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js')
```

HTML script tag

Example of Integration with Highlight.js

Glimmer Sample Selector uses Highlight.js to provide Syntax Highlighting

In application.html.erb layout, it includes the following lines:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/styles/github.min.css">
<script src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js"></script>
```

In component code, it invokes `window.hljs` JS functions directly:

```
def highlight_code
  @code_element.dom_element.removeAttr('data-highlighted')
  # $$ enables interacting with global JS scope, like top-level hljs variable to use highlight.js library
  $$.hljs.highlightAll
  $$.hljs.initLineNumbersOnLoad
  scroll_to_glimmer_code
end
```

Glimmer DSL for Web Rails 7 Sample App

Run a sample or view a sample's code.

- Hello, World!
- Hello, Button!
- Hello, Form!
- Hello, Observer!
- Hello, Observer (Data-Binding)!
- Hello, Data-Binding!
- Hello, Content Data-Binding!
- Hello, Component!
- Hello, Component Slots!
- Hello, Component Listeners!
- Hello, Component Listeners (Default Slot)!
- Hello, Paragraph!
- Hello, Input Date/Time!
- Hello, Form (MVP)!
- Hello, Style!
- Hello, SVG!
- Button Counter
- Todo MVC
- Hello, glimmer_component Rails Helper! (Run | Code)
- Contact Manager (Run | Code)

Run

```
1   require 'glimmer-dsl-web'
2
3   class NumberHolder
4     attr_accessor :number
5
6     def initialize
7       self.number = 50
8     end
9   end
10
11  class HelloObserverDataBinding
12    include Glimmer::Web::Component
13
14    before_render do
15      @number_holder = NumberHolder.new
16    end
17
18    markup {
19      div {
20        div {
21          input(type: 'number', min: 0, max: 100) {
22            value <=> [@number_holder, :number]
23          }
24        }
25        div {
26          input(type: 'range', min: 0, max: 100) {
27            value <=> [@number_holder, :number]
28          }
29        }
30      }
```

Glimmer DSL for Web Performance

Glimmer DSL for Web can render 1000 elements in ~0.055 of a second on a MacBook Pro 14" 2023 (M2 Chip). That is more than fast enough for most business applications as it is way below the 0.500s-1.000s range recommended for websites.

https://sample-glimmer-dsl-web-rails7-app-black-sound-6793.fly.dev/

Glimmer DSL for Web Rails Sample Selector App

Glimmer DSL for Web Rails 7 Sample App

Run a sample or view a sample's code.

- Hello, World!
- Hello, Button!
- Hello, Form!
- Hello, Observer!
- Hello, Data-Binding!
- Hello, Content Data-Binding!
- Hello, Component!
- Hello, Paragraph!
- Hello, Input Date/Time!
- Button Counter
- Hello, glimmer_component Rails Helper! (Run | Code)

Run

```
1   require 'glimmer-dsl-web'
2
3   include Glimmer
4
5   Document.ready? do
6     div {
7       h1('Contact Form')
8
9       form {
10        div {
11          label('Name: ', for: 'name-field')
12          @name_input = input(type: 'text', id: 'name-field', required: true, autofocus: true)
13        }
14
15        div {
16          label('Email: ', for: 'email-field')
17          @email_input = input(type: 'email', id: 'email-field', required: true)
18        }
19
20        div {
21          input(type: 'submit', value: 'Add Contact') {
22            onclick do |event|
23              if ([@name_input, @email_input].all? {|input| input.check_validity })
24                event.prevent_default
25                # re-open table content and add row
26                @table_content {
```

Benefits of Glimmer DSL for Web

Benefits of Glimmer DSL for Web

With Glimmer DSL for Web, programmers do not waste time unnecessarily splitting their resources across multiple languages as they focus on developing the Frontend and Backend with one very advanced and expressive language, Ruby, so they end up with less code to maintain overall, which improves productivity and maintainability.

Benefits of Glimmer DSL for Web

With Glimmer DSL for Web, programmers become happy by using Opal Ruby due to avoiding JavaScript badly engineered, over-engineered, and premature-optimization-obsessed libraries (e.g. React, Angular, Vue, Ember, and Svelte) and not fighting JavaScript build issues (e.g. webpack) anymore.

Benefits of Glimmer DSL for Web

With Glimmer DSL for Web, customers are provided with a better user experience because programmers have the ability to reuse Backend Ruby logic in the Frontend as is, so that removes the need to make a lot of HTTP web requests, which speeds up the response time of user actions in the Frontend.

Benefits of Glimmer DSL for Web

With Glimmer DSL for Web, web application frontends are completed in about half the time and cost for customers (or much less, like 1/10 the time), which gives businesses a very big competitive advantage as they can finish 12 months of Frontend Development work in about 6 months using Ruby in the Frontend.

Benefits of Glimmer DSL for Web

Save AI credits by generating very terse Ruby code instead of JavaScript.

When I had AI regenerate a Glimmer Web Component as a React component, it became 10x its size while mixing many conflicting concerns in the same file, thus becoming more expensive to maintain in JavaScript.

By the way, AI can already generate fully working Glimmer Web Components from scratch for previously unimplemented usecases even though Glimmer DSL for Web is still a very new project.

Benefits of Glimmer DSL for Web

With Glimmer DSL for Web, companies cut their hiring budget in half as Backend Ruby Developers do Frontend development in Ruby, so they become Full-Stack Developers who produce better products for clients.

Future of Glimmer DSL for Web

- Server-Side Rendering of Frontend Web Components as a replacement of ERB with automatic hydration
- GRB (Glimmer RB) is a Glimmer Ruby backend templating language alternative to ERB
- Connect Frontend Models with Backend Models in a Ruby on Rails application via Proxy objects (Proxy Pattern), bypassing the need for REST APIs completely with a lot less code in Isomorphic Ruby.

Future of Glimmer DSL for Web

- Consider supporting Ruby WASM as an alternative to Opal
- Explore a DSL for Web Canvas API
- Explore how to integrate with WebSockets
- Support HTML Web Components standard (enables reuse of React components or components built in other JavaScript Frontend libraries)

Q & A

- **Glimmer DSL for Web GitHub:** ⭐ https://github.com/AndyObtiva/glimmer-dsl-web
- **Glimmer Web Components GitHub:** ⭐ https://github.com/AndyObtiva/glimmer-web-components
- **My LinkedIn:** www.linkedin.com/in/andymaleh
- **My GitHub:** github.com/AndyObtiva